Secure Pitch Booking

Darius Mureșan · Student no. 202502066

Aarhus University

1 Introduction

This report presents the process of building a small and security-focused booking system that facilitates reserving and managing minifootball pitches across Brașov, a city in the heart of Romania. Its users are split into two main categories: player and admin. A player can view a full list of available pitches, book them (or cancel his own bookings) and request a weekly schedule of his upcoming matches. The administrator is the one that manages the entire platform: he can add (or remove) pitches, see every booking and has the permission to cancel any booking, but he can't create one. As minifootball continues growing in Romania, booking is a process worth automating and, by doing so, the system has to respect a straightforward ownership rule: every booking and every schedule belongs to one player and no other user should be able to reach data that does not belong to him.

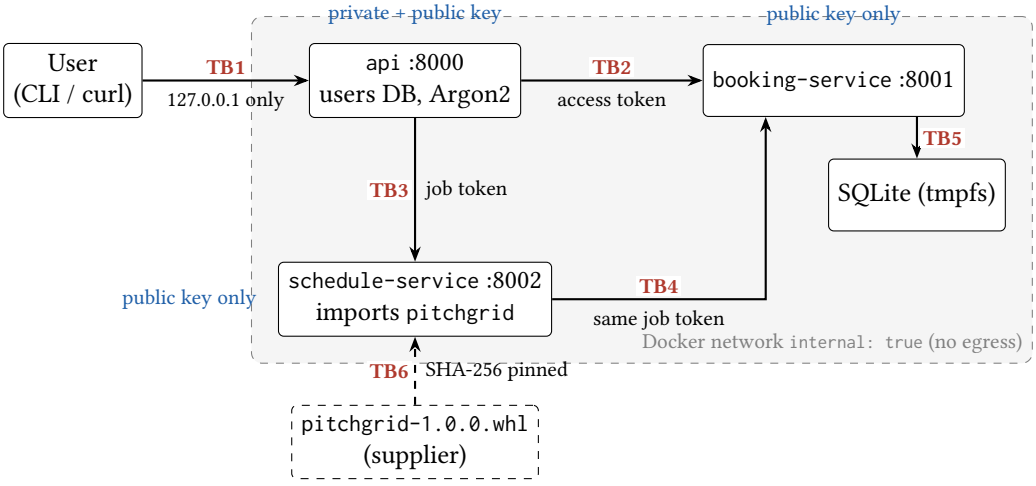
The system is split into three microservices with one external package. The `api` service authenticates the user and issues a signed token with their role, `booking-service` owns the data and decides what each user may do with it and `schedule-service` builds the weekly schedule using `pitchgrid`, a small package that is treated as third-party code, and receives only the access that a single job needs.

The main focus of this system was not performance, but to create an environment where each component has different responsibilities and permissions. Authority, therefore, has to cross service boundaries, and that is where the interesting problems appear, including one vulnerability that exists in no single service on its own. The source code is available at `https://github.com/dariusmuresan01/secure-pitch-booking`.

2 Architecture

The boundaries follow the three questions every request raises, namely, "who is asking?" (`api`), "may they do this to this object?" (`booking-service`) and "what untrusted computation is needed"? (`schedule-service` with `pitchgrid`). Keeping the decision in the service that owns the data means a bug in the gateway cannot grant access on its own, and isolating the third-party code in a service with no database means a compromised package inherits as little as possible. The weekly schedule was chosen as the central workflow because it is the only one that touches all three services and the package, so authority must travel two hops before anything is read.

Component	Responsibility	Deliberately not given
`api`	Checks passwords (Argon2), issues signed tokens and forwards requests. The only service published, and only on `127.0.0.1`.	No access to bookings or schedules; makes no decisions about them.
`booking-service`	Owns the SQLite database; checks role, ownership and job scope on every request.	No signing key: can verify tokens, never create them.
`schedule-service`	Builds one weekly schedule with `pitchgrid`.	No database, no user tokens, no signing key; only the job token of the current job.
`pitchgrid`	Turns a list of bookings into text.	Receives only the bookings of the job.



Trust is established in four distinct places, each one in exactly one component:

- **Identity** is established only at `POST /login` in `api`, by verifying an Argon2 hash from a users database that no other service can see (`32d8989`).
- **Authority** lives in EdDSA (Ed25519) tokens signed with a private key that only the `api` container mounts (`25bad91`, `3a96113`, `3855a9d`).
- **Ownership** is decided in `booking-service`, which stores `user_id`/`owner_id` from the token's `sub` claim and compares it on every read and write (`7a58baf`, `e79a8a6`).
- **Dependency trust** is established by the consumer: `schedule-service/requirements.txt` pins `pitchgrid==1.0.0` to a SHA-256 digest, and `scripts/verify_dependencies.py` recomputes the digest of every wheel in `packages/` before the build (`aaab364`, `6f6afa5`).

3 Authority Through the Workflow

Authority is carried by two JWT types, signed with Ed25519, that share the same signing key but cannot be substituted for one another (`7691012`). Every verifier pins the algorithm to `EdDSA`, the issuer to `api`, requires all listed claims and checks `token_use` explicitly.

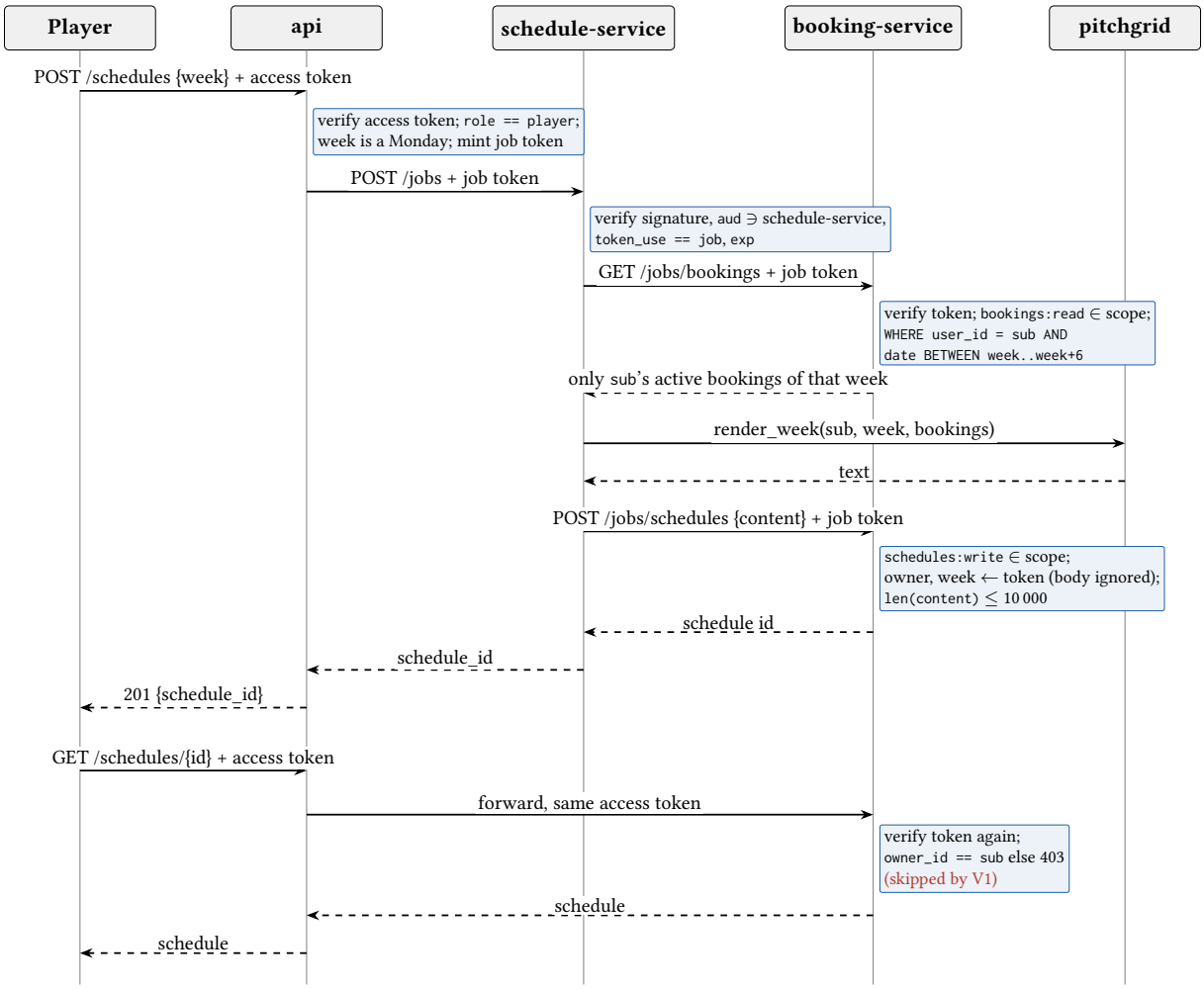
Claim	**Access token** (user, 15 min)	**Job token** (one schedule job, 2 min)
`sub`	username	the player who requested the job
`token_use`	`access`	`job`
`aud`	`booking-service`	`schedule-service`, `booking-service`
`role` / `scope`	`player` or `admin`	`bookings:read`, `schedules:write`
`week`	–	ISO date of the Monday

For ordinary requests the path is short. `api` verifies the access token, which keeps anonymous traffic away from the internal services, and forwards the same token. `booking-service` then verifies it again and derives both the role and the owner from its claims, never from the request body or the URL; a booking created with `"user_id": "andreea"` in the body is still owned by the caller (`test_booking_owner_is_taken_from_token_not_body`).

The admin follows exactly the same path. His role is written in the token by `api` at login, so `booking-service` never has to ask `api` who he is; it only reads the `role` claim from a token that it has verified itself. Since the role cannot be changed without breaking the signature, a

player who edits his token to say `admin` is rejected with 401 before any check on the data is made (`test_tampered_token_is_rejected`).

Next figure follows operation 5. The player's access token never leaves the `api` - `booking-service` path: instead of forwarding the user's full authority to `schedule-service`, `api` mints a new token that names one player, one week and two actions. The second half of the flow, the download, is an ordinary user request and is mediated by the ownership check.



`schedule-service` is a permitted caller of `booking-service`, so the interesting question is what stops it from doing more than its job. Each limit below is enforced by `booking-service` or by key distribution, not by the good behaviour of the caller, and each one is pinned by a test in `tests/internal/test_job_tokens.py` (`ae072fe`), which runs inside the internal network and plays the attacker:

- **It cannot mint authority.** Only `api` holds the private key; a token signed with any other key is rejected by both services (`test_*_rejects_job_token_signed_with_unknown_key`).
- **It cannot exceed the job.** Rows are filtered by `sub` and `week` from the token, and each endpoint requires its own scope (`test_job_token_reads_only_its_player_and_week`, `test_job_token_without_read_scope_is_forbidden`, `test_job_token_without_write_scope_cannot_save_schedule`).
- **It cannot choose the owner.** `owner_id` and `week` of a saved schedule come from the token even if the body names someone else, and oversized content is rejected (`test_saved_schedule_owner_comes_from_job_token`, `test_oversized_schedule_is_rejected`).

- **It cannot reuse a token for something else.** A job token is refused on user endpoints, a user token is refused on job endpoints, and a token whose audience lacks `schedule-service` is refused by it (`test_job_token_cannot_be_used_as_user_token`, `test_user_token_cannot_be_used_as_job_token`, `test_job_token_not_meant_for_schedule_service_is_rejected`).
- **Its authority expires.** A job token older than two minutes is rejected (`test_expired_job_token_is_rejected`).

The same reasoning applies to `api`: `booking-service` does not trust it simply because it sits on the internal network, and re-verifies every forwarded access token (`test_booking_service_rejects_requests_without_token`). One limitation remains: the job token is a bearer token that `schedule-service` forwards unchanged, so within its two minutes it could be replayed to save further schedules for the same player and week. Section 7 returns to this.

4 Secure Design Principles

The plan names three of Saltzer and Schroeder's principles, and the table below maps each to the code and to the evidence that it holds.

Principle	Where it appears	Evidence
Least privilege	Signing key only in `api`; per-service key mounts; `schedule-service` has no DB volume; job token limited to one player, one week, two scopes, two minutes; containers run as UID 10001 with `cap_drop: ALL`, `read_only`, `no-new-privileges`; admin cannot book or read schedules.	`docker-compose.yml` (`3855a9d`); `test_admin_cannot_create_booking`, `test_admin_cannot_download_player_schedule`, job token tests.
Complete mediation	`booking-service` verifies the token and checks role/ownership/scope on every endpoint, including internal ones; the gateway makes no authorization decisions about data.	`booking-service/main.py`; `test_anonymous_requests_are_rejected` (9 routes); `test_tampered_token_is_rejected`; all 403 tests in `tests/test_authorization.py` (`61e8ce1`).
Fail-safe defaults	Missing/invalid token ⇒ 401; anything not in the access matrix ⇒ 403; Pydantic ranges and an allow-list for `status`; vulnerable code paths enabled only if `VULNERABLE_MODE` is exactly `"1"`.	`test_malformed_booking_is_rejected` (5 cases); `test_invalid_week_start_is_rejected`; build #1: 55/55 pass in default mode.

Least privilege was applied to people as well as to services. Already at Gate 1 the access matrix denied the admin booking and reading schedules: an administrator who can book in a player's name, or read a player's schedule, would be trusted with something the platform does not require. When the admin was later redefined from pitch owner to platform administrator (`fc620bb`), pitch ownership moved to the system and no right was added. Complete mediation is the reason the gateway is deliberately thin: `api` only verifies that a token is genuine and forwards it, so the decision is made once, where the data is. The fail-safe flag is a small but deliberate choice: `os.environ.get("VULNERABLE_MODE") == "1"` means a typo, an empty value or a missing variable all produce the secure build.

The three principles also explain why V4 is possible. The job token is a good example of least privilege, but in vulnerable mode `booking-service` stops mediating on the `sub` claim, so the narrow authority written in the token is simply ignored. A narrow token is only useful if the receiver actually enforces every claim in it.

5 Planted Vulnerabilities

Four vulnerabilities were added after Gate 3, behind the single `VULNERABLE_MODE` switch (`fd2f3f8`, `2f88210`), and documented in `docs/known-vulnerabilities.md` using CWE identifiers. Each has a demonstration test in `tests/vulnerabilities/` that asserts the attack fails in secure mode and succeeds in vulnerable mode, so it passes in both. Jenkins stage 8 restarts the stack with `docker-compose.vulnerable.yml` and runs these tests (`046feb1`); build #5 shows all five passing (screenshot, `aa3aa67`).

Id	Weakness	Location	Impact	Detected by
V1	CWE-862 Missing Authorization (T10)	`get_schedule`: ownership check skipped	Any user reads any schedule by id	2 plan tests (stage 6); not Bandit
V2	CWE-89 SQL Injection (T6)	`get_bookings`: `status` concatenated into SQL	`x' OR '1'='1` returns all players' bookings	Demo test only
V3	CWE-20 Improper Input Validation (no threat)	`create_booking`: past dates accepted	Fabricated bookings in the past	Demo test only
V4	Composition (T11)	`/jobs/bookings` + `schedule-service`	A player's schedule lists other players' bookings	2 plan tests (stage 6); not Bandit

V1 removes one comparison, `schedule["owner_id"] != user["sub"]`. Because schedule ids are sequential, guessing them is trivial. **V2** introduced a new feature, the `status` filter, and in vulnerable mode appends it to the query as a string instead of a bound parameter, while secure mode both binds it and checks it against {`active`, `cancelled`}. Bandit's rule B608 did not flag it: the statement is assembled from fragments and an `if` branch, which its pattern matching does not follow. **V3** is a semantic validation gap: the date is well formed, so Pydantic accepts it, but nothing asks whether a booking in the past makes sense. Notably, no STRIDE threat covered this, so the plan had no test for it either.

In practice, exploiting V1 needs nothing more than a valid login: Andreea logs in normally, requests her own schedule to see roughly where the ids are, and then simply counts down. Every response is a 200 that contains the owner, the week and the full timetable of another player, which is exactly the data the ownership rule from the introduction was supposed to protect.

V4 is the composition vulnerability and the most interesting one, because both services are individually defensible. `booking-service` treats `/jobs/bookings` as a weekly feed for trusted jobs and scopes it only by week; `schedule-service` assumes the list it receives is already limited to the job's player and passes it to `pitchgrid` unchanged. Each assumption is reasonable in isolation; together, no one filters by player. A review of either diff alone would plausibly approve it; only a threat that names the assumption (T11) and an end-to-end test that crosses both services reveal it:

```
# booking-service, vulnerable branch of get_job_bookings
"SELECT id, pitch_id, date, start_hour FROM bookings "
"WHERE status = 'active' AND date BETWEEN ? AND ?"     # no user_id = ?
```

The two tests that caught V1 and V4 were written from the access matrix before the vulnerabilities existed, which is the strongest evidence that deriving tests from the plan works for authorization. V2 and V3 were not caught by any earlier test, because the features they abuse (a status filter, a past-date rule) were not in the plan; running the whole suite in vulnerable mode gives 4 failures and 56 passes (README). Static analysis detected none of the four.

6 Lifecycle, Gates and Protections

The plan was written and reviewed before any code: its sections were committed one by one on 8 October (`007b447 - 651cedd`) and Gate 1 was tagged on that commit (`gate-1-requirements`); the first service code followed on 10 October. The plan then shaped development in three concrete ways. First, the traceability table turned each STRIDE control into a commit: T3 into Ed25519 keys (`25bad91`), T5 into "owner from token" (`e79a8a6`), T11 into the job-scoped endpoint (`d67d9a9`), T9 into audit logging (`cc8e02b`). Second, the rule that "each no in the access matrix has a test" produced `tests/test_authorization.py` almost mechanically. Third, the pipeline stages in the plan became the `Jenkinsfile` almost one to one, with two additions. Stage 8 runs the vulnerability demonstrations, and stage 7 greps the service logs and fails if a token (`eyJ`) appears or if no 403 was logged, so T9 and T12 are checked automatically.

Gate	Tag / commit	Decision and evidence
G1 Requirements	`gate-1-requirements` / `651cedd`	Scope, assets A1–A10, TB1–TB6, access matrix and 15 threats each with a control.
G2 Build	`gate-2-build` / `e7618d3`	Jenkins build #1: all requirements pinned with hashes, wheel digest matches, Bandit no high findings, Gitleaks no secrets, pip-audit no known vulnerabilities (exceptions file empty).
G3 Testing	`gate-3-testing` / `e7618d3`	Build #1: 55/55 tests (42 black-box, 13 internal), no tokens in logs (`bd895b8`).

The record is honest about one weakness of the process: G2 and G3 were tagged on the same commit, seconds apart, because the pipeline was finished only after the tests. In practice the gates were evaluated retroactively by one Jenkins build instead of blocking each phase as it ended. A flaky test (`8677554`) was fixed before the gate was claimed rather than retried until green.

The dependency gate also changed how dependencies were added. Every `requirements.txt` is generated from a short `requirements.in` with hashes for each package, so adding a library meant regenerating and committing the whole file. The same holds for `pitchgrid`: any change to the wheel changes its digest, so a new version is accepted only after it is approved explicitly in `schedule-service/requirements.txt`.

The gates also constrained what came after them. Since G2 and G3 certified a clean baseline, the planted vulnerabilities had to be added without making the pipeline lie: they are off by default, so stage 6 still fails if one leaks into the secure build, and stage 8 fails if a later change silently removes one. This matches the plan's rule that planted vulnerabilities are the only accepted exceptions, each listed together with the test that demonstrates it.

Layer	Protections
Language / runtime	Python is memory safe; type hints and Pydantic models give type, length and range checks with automatic 422 responses; `sqlite3` parameter binding.
Libraries	`argon2-cffi` for salted, memory-hard hashes; PyJWT with an algorithm allow-list and required claims; `cryptography` for Ed25519 keys; `httpx` with timeouts.
Application logic	Role, ownership and scope checks; owner from token; status allow-list and past-date check; DB `CHECK` constraints and a partial unique index against double booking; identical login errors; JSON audit log without tokens.
Deployment	Only `api` published, on `127.0.0.1`; internal network without egress; non-root, read-only containers with no capabilities; one key per service, read-only mounts; keys never committed and regenerated per CI build; `pip install --require-hashes`.

7 Towards a Production Release

The prototype is safe to run locally, not to deploy. Most of what is missing is a stronger version of a control that already exists, rather than a completely new one. A production release would need at least:

- **Remove the vulnerable code paths** from the release build instead of relying on a flag; keep the demonstrations on a separate branch.
- **Transport security and service identity**: TLS at the edge and mutual TLS between services, since tokens currently travel in plain HTTP inside the Docker network.
- **Key management**: a secrets manager, key rotation with a `kid` header, and no private key in the test container.
- **Narrower delegation**: token exchange so that each hop receives a token for exactly one audience, and a `jti` that makes job tokens single use; a revocation path for access tokens.
- **Abuse controls**: rate limiting and lockout on `/login` and on schedule jobs, plus request-size limits; T1 and T13 are only partly covered today.
- **Durable data**: a managed database with backups and migrations instead of SQLite in `tmpfs`, and centralised, tamper-evident logs that include `schedule-service`.
- **Hardened pipeline**: Jenkins currently runs as root with the Docker socket mounted, which is equivalent to root on the host; use ephemeral agents, pin base images by digest, sign images, publish an SBOM, and make gates block merges through branch protection.
- **Real accounts**: registration, a password policy and multi-factor login for the admin, replacing the synthetic seeded accounts whose passwords are in the repository.
- **Better analysis**: V2 shows that pattern-based SAST is not enough; taint-tracking analysis (e.g. CodeQL or Semgrep rules) and fuzzing of the query parameters would be added.